

Nexus ERP Fase 3

Para que você comece a desenvolver agora mesmo com um **Banco de Dados Real** e o **Backend**, a melhor forma é usar o **Docker**. Ele permite que você suba um banco de dados PostgreSQL profissional em segundos, sem precisar instalar nada manualmente no seu computador.

Aqui está o **Script de Automação de Ambiente** para o seu Nexus ERP.

Passo 1: O Arquivo de Configuração do Banco (`docker-compose.yml`)

Crie um arquivo chamado `docker-compose.yml` . Ele vai configurar o seu PostgreSQL real.

``yaml

Version: '3.8'

Services:

Db:

Image: postgres:15

Container_name: nexus_db

Restart: always

Environment:

POSTGRES_USER: admin

POSTGRES_PASSWORD: nexuspassword

POSTGRES_DB: nexus_erp

Ports:

- "5432:5432"

Volumes:

- nexus_data:/var/lib/postgresql/data

Volumes:

Nexus_data:

...

Passo 2: O Script de Instalação de Dependências

Crie um arquivo chamado `setup\_env.sh` (para Linux/Mac) ou `setup\_env.bat` (para Windows).

****Para Windows (setup_env.bat):****

```
```batch
```

```
@echo off
```

```
Echo Instalando dependencias do Nexus ERP...
```

```
Pip install fastapi uvicorn sqlalchemy psycpg2-binary pydantic
```

```
Echo Dependencias instaladas.
```

```
Echo Subindo Banco de Dados Real via Docker...
```

```
Docker-compose up -d
```

```
Echo Ambiente Pronto!
```

```
```
```

Passo 3: O Código do Backend Conectado (`main.py`)

Este código já cria as tabelas de ****Compras, Vendas e RH**** assim que você o executa.

```
```python
```

```
from fastapi import FastAPI, Depends, HTTPException
```

```
from sqlalchemy import create_engine, Column, String, Float, DateTime,
ForeignKey
```

```
From sqlalchemy.ext.declarative import declarative_base
```

```
From sqlalchemy.orm import sessionmaker, Session
```

```
Import datetime
```

```
Import uuid
```

```
1. CONEXÃO COM O BANCO REAL (DOCKER)
```

```
DATABASE_URL =
```

```
“postgresql://admin:nexuspassword@localhost:5432/nexus_erp”
```

```
Engine = create_engine(DATABASE_URL)
```

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

```
Base = declarative_base()
```

```
2. MODELOS DO BANCO DE DADOS (O Triângulo 100%)
```

```
Class Product(Base):
```

```
__tablename__ = “products”
```

```
Id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
```

```
Name = Column(String)
```

```
Price = Column(Float)
```

```
Stock = Column(Float)
```

```
Class Sale(Base):
```

```
__tablename__ = “sales”
```

```
Id = Column(String, primary_key=True, default=lambda: str(uuid.uuid4()))
```

```
Customer = Column(String)
```

```
Total = Column(Float)
```

```
Created_at = Column(DateTime, default=datetime.datetime.utcnow)
```

```
Criar tabelas no Banco
```

```
Base.metadata.create_all(bind=engine)
```

### # 3. APP E ENDPOINTS

```
App = FastAPI(title="Nexus ERP Real-Time")
```

```
Def get_db():
```

```
 Db = SessionLocal()
```

```
 Try:
```

```
 Yield db
```

```
 Finally:
```

```
 Db.close()
```

```
@app.get("/")
```

```
Def status():
```

```
 Return {"status": "Nexus ERP Online", "database": "Connected"}
```

```
@app.post("/sales/new")
```

```
Def register_sale(customer: str, value: float, db: Session = Depends(get_db)):
```

```
 Nova_venda = Sale(customer=customer, total=value)
```

```
 Db.add(nova_venda)
```

```
 Db.commit()
```

```
 Return {"message": "Venda registrada no banco real!", "id": nova_venda.id}
```

```
@app.get("/inventory/list")
```

```
Def list_products(db: Session = Depends(get_db)):
```

```
 Return db.query(Product).all()
```

```
...
```

---

### ### Como Rodar Tudo em 3 Passos:

1. **Instale o Docker Desktop** (se ainda não tiver).
  2. **Abra o Terminal** na pasta onde você criou os arquivos.
  3. **Execute os comandos:**
    - \* ``docker-compose up -d`` (Isso liga o banco de dados PostgreSQL).
    - \* ``uvicorn main:app --reload`` (Isso liga o seu sistema ERP).
- 

### ### Por que isso é “Nível Profissional”?

1. **Persistência Real:** Você pode desligar o PC, e quando ligar o Docker, as vendas e produtos do seu Nexus estarão lá.
2. **Escalabilidade:** Esse mesmo ``main.py`` pode ser colocado na nuvem (AWS/Azure) amanhã sem mudar quase nada.
3. **Ambiente Isolado:** O banco de dados não “suja” o seu Windows; ele fica dentro do container Docker, garantindo a **Qualidade**.

### ### Próximo Passo:

Com o banco rodando e o backend respondendo, você quer que eu te ajude a criar o **primeiro formulário de frontend** (uma página simples) para você clicar e ver os dados aparecendo no banco?